

Chimney Damage Detection

Deep Learning Project using YOLOv8 Segmentation

Presented By

MMM

Presented To

Amit B. Shelke



Problem & Objective

The Problem

Manual chimney inspection is **time-consuming, hazardous, and error-prone**. Human inspectors struggle to consistently identify subtle structural damage across large structures.

- ❏ Pixel-level segmentation outperforms bounding-box detection by precisely outlining damaged regions — enabling accurate damage quantification.

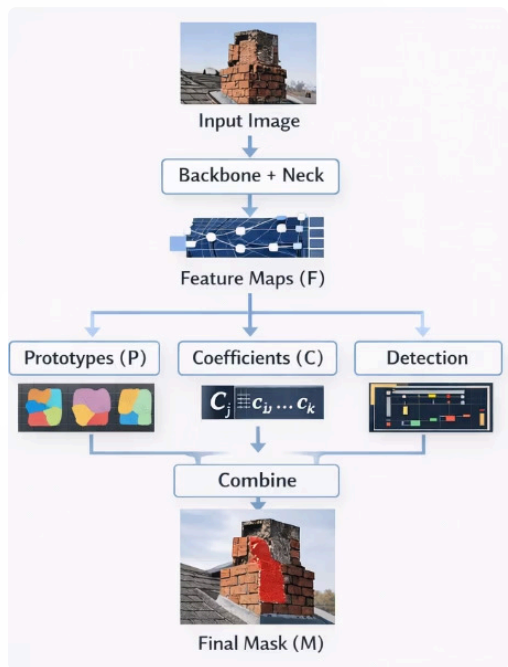
Project Objectives

- Build a YOLOv8 segmentation model
- Detect multiple damage classes
- Evaluate model performance rigorously



YOLOv8 Segmentation Model

ARCHITECTURE OVERVIEW



You Only Look Once (YOLO)

YOLOv8 is the **latest generation** with improved segmentation heads, anchor-free detection, and superior performance on custom domain tasks.

Mathematical Formulation

- ◆ Feature Extraction $F \in \mathbb{R}^{H' \times W' \times C}$
- ◆ Prototype Masks $P \in \mathbb{R}^{H' \times W' \times k}$
- ◆ Mask Coefficients (per object) $C(j) \in \mathbb{R}^k$
- ◆ Final Mask Generation $M_j = \sigma\left(\sum_{i=1}^k C_j \cdot P_i\right)$



Real-Time Speed



High Accuracy



Custom Datasets

Dataset & Annotation

480

Total Images

4

Damage Classes

80%

Train Split

20%

Val Split



Damage



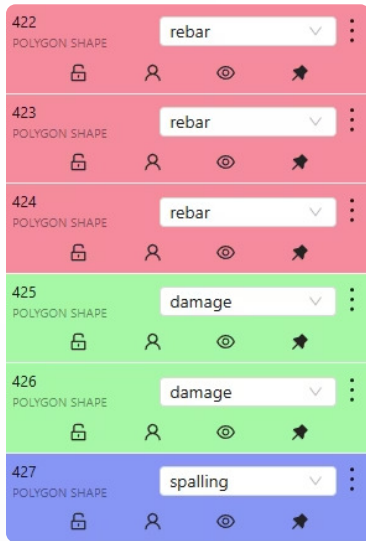
Spalling



Honeycombing



Exposed Rebars



Annotation Tool: CVAT

Polygon annotations were drawn around each damage region using **CVAT (Computer Vision Annotation Tool)** for precise, pixel-accurate segmentation masks.



Google Docs

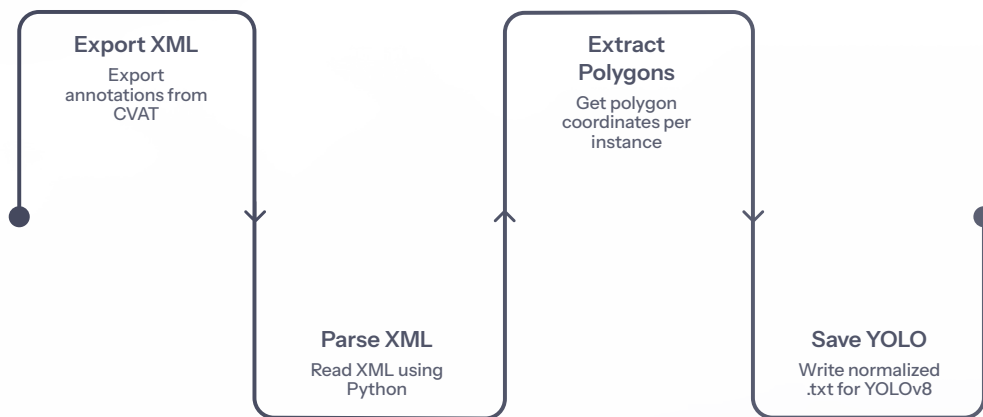


annotations.xml

Data Preparation

CVAT → YOLO FORMAT CONVERSION

Conversion Pipeline



Each label file stores normalized polygon coordinates per line — one row per damage instance — compatible with YOLOv8's segmentation trainer.

Final Dataset Structure

 **images/**

train/ val/

 **labels/**

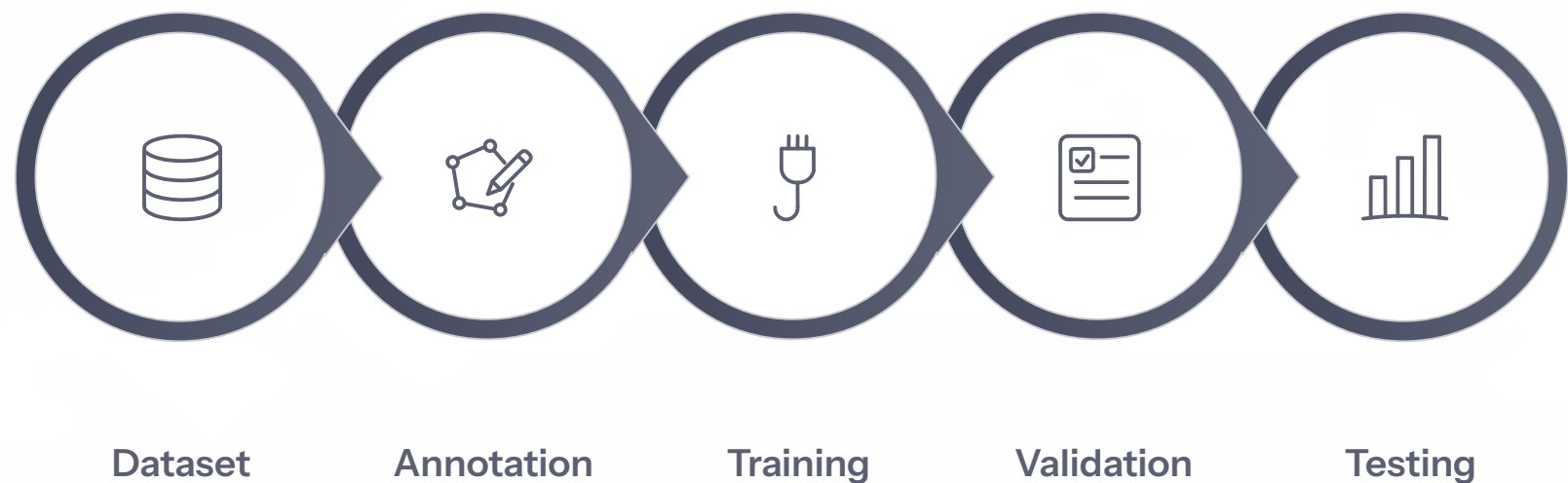
train/ val/

 **data.yaml**

Class names & dataset paths

- ❏ The CVAT XML file encodes label names and polygon vertex coordinates per image, which are parsed using Python's `xml.etree` library.

Methodology



Model

YOLOv8n-seg — nano variant optimized for efficient segmentation on limited hardware.

Training Configuration

 Image Size: 640	 Epochs: 50
 Batch Size: 4	 Device: CPU

Evaluation Metrics



mAP50

Mean Average Precision at **50% IoU threshold**. Measures how accurately the model detects and segments damage regions overall.



Precision

Ratio of **correct positive predictions** to all predictions made. High precision means fewer false alarms.



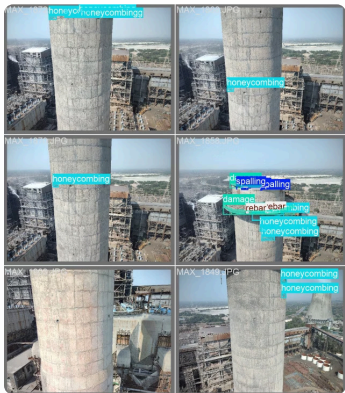
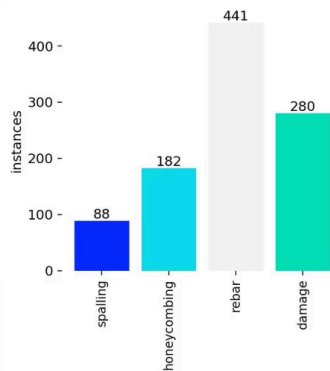
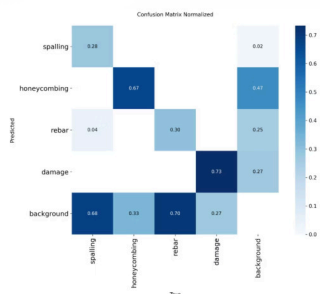
Recall

Ratio of **detected objects** to all actual objects present. High recall means fewer missed detections.



F1 Score

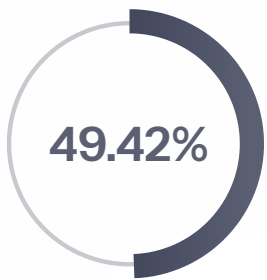
F1 Score gives a **balanced** measure of how **accurate and complete** the model's predictions are.



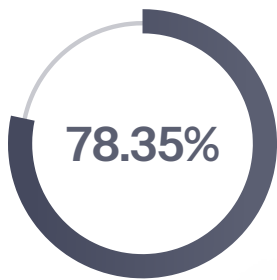
Results

50 EPOCHS · YOLOV8N-SEG · CPU TRAINING

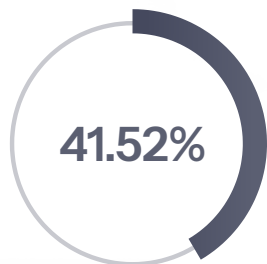
Model Performance



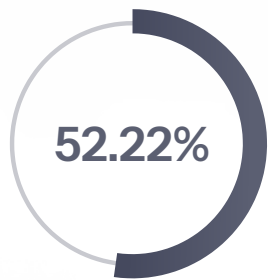
mAP50



Precision

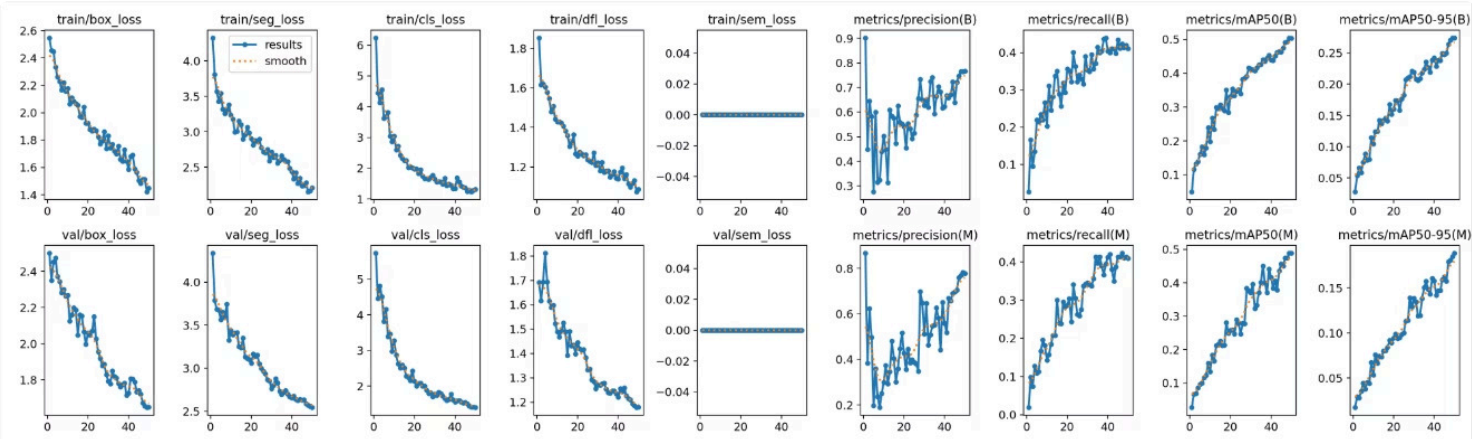


Recall



F1 Score

☐ The model successfully detects all four damage categories — demonstrating strong generalization on a compact 480-image dataset.



Visual Outputs

SEGMENTATION PREDICTIONS ON TEST IMAGES

Predicted segmentation masks accurately outline damage regions — demonstrating the model's ability to localize **Spalling**, **Honeycombing**, **Exposed Rebars**, and **general Damage** at pixel level.



Test Input Image

Original chimney image fed to the model during inference.



Annotated Image

The Chimney Image annotated by CVAT (Computer Vision Annotation Tool) with polygon annotations, represents actual damage regions.



Prediction

Segmentation masks predicted by YOLOv8 model. Used to compare with ground truth and evaluate performance.

Key Insights & Conclusion

Key Insights

✓ Pixel-level segmentation improves damage localization over bounding boxes

✓ YOLOv8 performs robustly even on a limited 480-image dataset

⚠ CPU-based training is significantly slower than GPU alternatives

📄 **Conclusion:** A functional chimney damage segmentation system was successfully built using YOLOv8 — proving deep learning viability for structural inspection automation.

Future Work

01

Expand dataset size for better generalization

02

Migrate training to GPU for faster iteration

03

Fine-tune hyperparameters and augmentation strategies

Thank You — Questions Welcome

